Chapter 1. The Basics

PostgreSQL is an extremely powerful piece of software that introduces features you may not have seen before. Some of the features are also present in other well-known database engines, but under different names. This chapter lays out the main concepts you should know when starting to attack PostgreSQL documentation, and mentions some related terms in other databases.

We begin by pointing you to resources for downloading and installing PostgreSQL. Next, we provide an overview of indispensable administration tools followed by a review of PostgreSQL nomenclature. PostgreSQL 10 was recently released. We'll highlight some of the new features therein. We close with resources to turn to when you need additional guidance and to submit bug reports.

Why PostgreSQL?

PostgreSQL is an enterprise-class relational database management system, on par with the very best proprietary database systems: Oracle, Microsoft SQL Server, and IBM DB2, just to name a few. PostgreSQL is special because it's not just a database: it's also an application platform, and an impressive one at that.

PostgreSQL is fast. In benchmarks, PostgreSQL either exceeds or matches the performance of many other databases, both open source and proprietary.

PostgreSQL invites you to write stored procedures and functions in numerous programming languages. In addition to the prepackaged languages of C, SQL, and PL/pgSQL, you can easily enable support for additional languages such as PL/Perl, PL/Python, PL/V8 (aka PL/JavaScript), PL/Ruby, and PL/R. This support for a wide variety of languages allows you to choose the language with constructs that can best solve the problem at hand. For

instance, use R for statistics and graphing, Python for calling web services, the Python SciPy library for scientific computing, and PL/V8 for validating data, processing strings, and wrangling with JSON data. Easier yet, find a freely available function that you need, find out the language that it's written in, enable that specific language in PostgreSQL, and copy the code. No one will think less of you.

Most database products limit you to a predefined set of data types: integers, texts, Booleans, etc. Not only does PostgreSQL come with a larger built-in set than most, but you can define additional data types to suit your needs. Need complex numbers? Create a composite type made up of two floats. Have a triangle fetish? Create a coordinate type, then create a triangle type made up of three coordinate pairs. A dozenal activist? Create your own duodecimal type. Innovative types are useful insofar as the operators and functions that support them. So once you've created your special number types, don't forget to define basic arithmetic operations for them. Yes, PostgreSQL will let you customize the meaning of the symbols (+,-,/,*). Whenever you create a type, PostgreSQL automatically creates a companion array type for you. If you created a complex number type, arrays of complex numbers are available to you without additional work.

PostgreSQL also automatically creates types from any tables you define. For instance, create a table of dogs with columns such as breed, cuteness, and barkiness. Behind the scenes, PostgreSQL maintains a dogs data type for you. This amazingly useful bridge between the relational world and the object world means that you can treat data elements in a way that's convenient for the task at hand. You can create functions that work on one object at a time or functions that work on sets of objects at a time. Many third-party extensions for PostgreSQL leverage custom types to achieve performance gains, provide domain-specific constructs for shorter and more maintainable code, and accomplish feats you can only fantasize about with other database products.

Our principal advice is this: don't treat databases as dumb storage. A database such as PostgreSQL can be a full-fledged application platform. With a robust database, everything else is eye candy. Once you're versant in SQL, you'll be able to accomplish in seconds what would take a casual

programmer hours, both in coding and running time.

In recent years, we've witnessed an upsurge of NoSQL movements (though much of it could be hype). Although PostgreSQL is fundamentally relational, you'll find plenty of facilities to handle nonrelational data. The ltree extension to PostgreSQL has been around since time immemorial and provides support for graphs. The hstore extensions let you store key-value pairs. JSON and JSONB types allow storage of documents similar to MongoDb. In many ways, PostgreSQL accommodated NoSQL before the term was even coined!

PostgreSQL just celebrated its 20th birthday, dating from its christening to PostgreSQL from Postgres95. The beginnings of the PostgreSQL code-base began well before that in 1986. PostgreSQL is supported on all major operating systems: Linux, Unix, Windows, and Mac. Every year brings a new major release, offering enhanced performance along with features that push the envelope of what's possible in a database offering.

Finally, PostgreSQL is open source with a generous licensing policy. PostgreSQL is supported by a community of developers and users where profit maximization is not the ultimate pursuit. If you want features, you're free to contribute, or at least vocalize. If you want to customize and experiment, no one is going to sue you. You, the mighty user, make PostgreSQL what it is.

In the end, you will wonder why you ever used any other database, because PostgreSQL does everything you could hope for and does it for free. No more reading the licensing cost fineprint of those other databases to figure out how many dollars you need to spend if you have eight cores on your virtualized servers with *X* number of concurrent connections. No more fretting about how much more the next upgrade will cost you.

Why Not PostgreSQL?

Given all the proselytizing thus far, it's only fair that we point out situations when PostgreSQL might not be suitable.

The typical installation size of PostgreSQL without any extensions is more than 100 MB. This rules out PostgreSQL for a database on a small device or as a simple cache store. Many lightweight databases abound that could better serve your needs without the larger footprint.

Given its enterprise stature, PostgreSQL doesn't take security lightly. If you're developing lightweight applications where you're managing security at the application level, PostgreSQL security with its sophisticated role and permission management could be overkill. You might consider a single-user database such as SQLite or a database such as Firebird that can be run either as a client server or in single-user embedded mode.

All that said, it is a common practice to combine PostgreSQL with other database types. One common combination you will find is using Redis or Memcache to cache PostgreSQL query results. As another example, SQLite can be used to store a disconnected set of data for offline querying when PostgreSQL is the main database backend for an application.

Finally, many hosting companies don't offer PostgreSQL on a shared hosting environment, or they offer an outdated version. Most still gravitate toward the impotent MySQL. To a web designer, for whom the database is an afterthought, MySQL might suffice. But as soon as you learn to write any SQL beyond a single-table select and simple joins, you'll begin to sense the shortcomings of MySQL. Since the first edition of this book, virtualization has resown the landscape of commerical hosting, so having your own dedicated server is no longer a luxury, but the norm. And when you have your own server, you're free to choose what you wish to have installed. PostgreSQL bodes well with the popularity of cloud computing such as Platform as a Service (PaaS) and Database as a Service (DbaaS). Most of the major PaaS and DbaaS providers offer PostgreSQL, notably Heroku, Engine Yard, Red Hat OpenShift, Amazon RDS for PostgreSQL, Google Cloud SQL for PostgreSQL, Amazon Aurora for PostgreSQL, and Microsoft Azure for PostgreSQL.

Where to Get PostgreSQL

Years ago, if you wanted PostgreSQL, you had to compile it from source. Thankfully, those days are long gone. Granted, you can still compile from source, but using packaged installers won't make you any less cool. A few clicks or keystrokes, and you're on your way.

If you're installing PostgreSQL for the first time and have no existing database to upgrade, you should install the latest stable release version for your OS. The downloads page for the PostgreSQL core distribution maintains a listing of places where you can download PostgreSQL binaries for various OSes. In Appendix A, you'll find useful installation instructions and links to additional custom distributions.

Administration Tools

Four tools widely used with PostgreSQL are psql, pgAdmin, phpPgAdmin, and Adminer. PostgreSQL core developers actively maintain the first three; therefore, they tend to stay in sync with PostgreSQL releases. Adminer, while not specific to PostgreSQL, is useful if you also need to manage other relational databases: SQLite, MySQL, SQL Server, or Oracle. Beyond the four that we mentioned, you can find plenty of other excellent administration tools, both open source and proprietary.

psql

psql is a command-line interface for running queries and is included in all distributions of PostgreSQL (see "psql Interactive Commands"). psql has some unusual features, such as an import and export command for delimited files (CSV or tab), and a minimalistic report writer that can generate HTML output. psql has been around since the introduction of PostgreSQL and is the tool of choice for many expert users, for people working in consoles without a GUI, or for running common tasks in shell scripts. Newer converts favor GUI tools and wonder why the older generation still clings to the command line.

pgAdmin

pgAdmin is a popular, free GUI tool for PostgreSQL. Download it separately from PostgreSQL if it isn't already packaged with your installer. pgAdmin runs on all OSes supported by PostgreSQL.

Even if your database lives on a console-only Linux server, go ahead and install pgAdmin on your workstation, and you'll find yourself armed with a fantastic GUI tool.

pgAdmin recently entered its fourth release, dubbed pgAdmin4. pgAdmin4 is a complete rewrite of pgAdmin3 that sports a desktop as well as a web server application version utilizing Python. pgAdmin4 is currently at version 1.5. It made its debut at the same time as PostgreSQL 9.6 and is available as part of several PostgreSQL distributions. You can run pgAdmin4 as a desktop application or via a browser interface.

An example of pgAdmin4 appears in Figure 1-1.

If you're unfamiliar with PostgreSQL, you should definitely start with pgAdmin. You'll get a bird's-eye view and appreciate the richness of PostgreSQL just by exploring everything you see in the main interface. If you're deserting Microsoft SQL Server and are accustomed to Management Studio, you'll feel right at home.

pgAdmin4 still has a couple of pain points compared to pgAdmin3, but its feature set is ramping up quickly and in some ways already surpasses pgAdmin3. That said, if you are a long-time user of pgAdmin3, you might want to go for the pgAdmin3 Long Time support (LTS) version supported and distributed by BigSQL, and spend a little time test-driving pgAdmin4 before you fully commit to it. But keep in mind that the pgAdmin project is fully committed to pgAdmin4 and no longer will make changes to pgAdmin3.

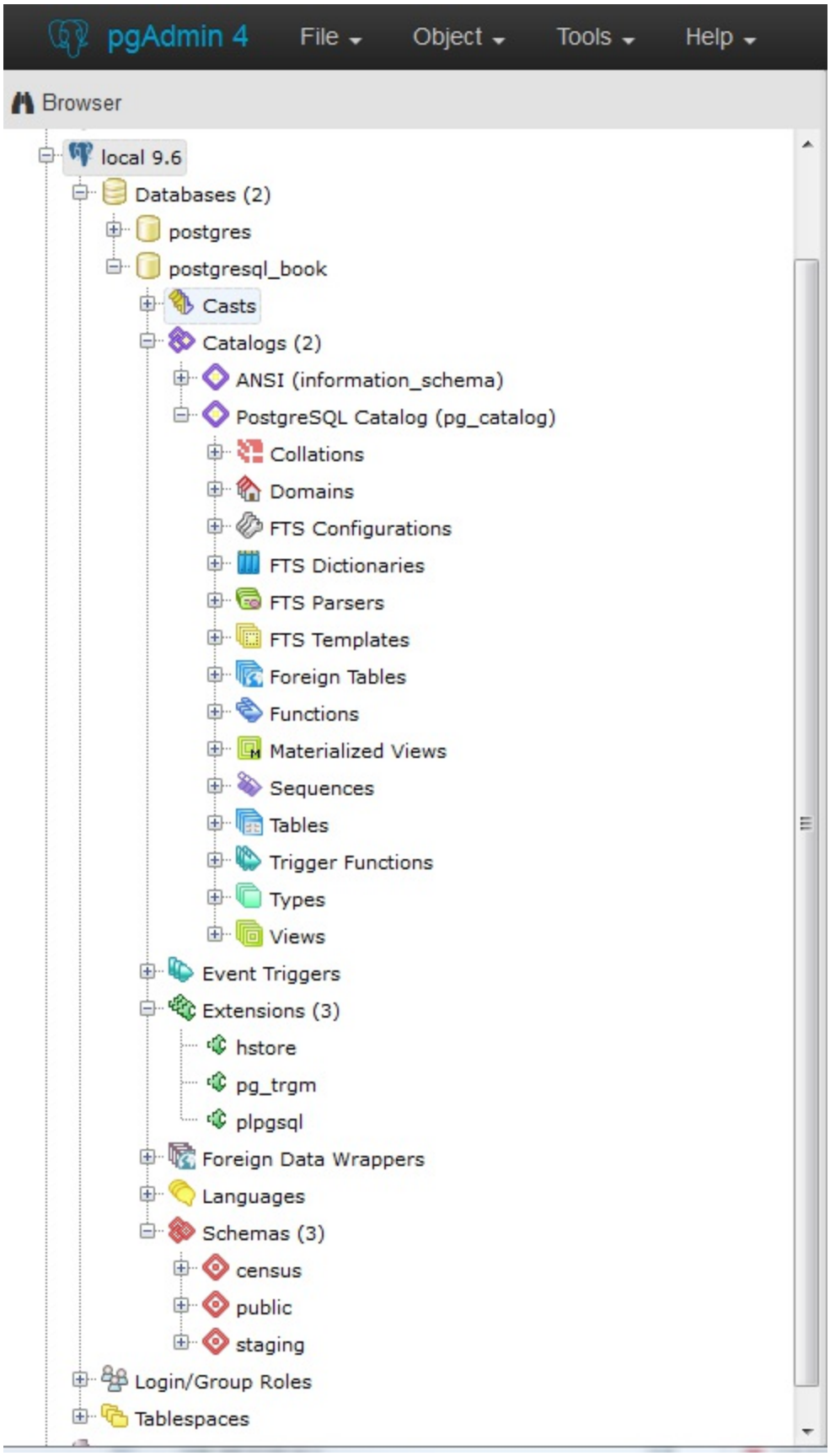
pgAdmin 4
File
Object
Tools
Help
Browser
local 9.6
Databases (2)
postgres
postgresql_book
Casts
Catalogs (2)
ANSI (information_schema)
PostgreSQL Catalog (pg_catalog)
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Sequences
Tables
Trigger Functions
Types
Views
Event Triggers
Extensions (3)
hstore
pg_trgm
plpgsql
Foreign Data Wrappers
Languages
Schemas (3)
census
public
staging
Login/Group Roles
Tablespaces

Figure 1-1. pgAdmin4 tree browser

phpPgAdmin

phpPgAdmin, pictured in Figure 1-2, is a free, web-based administration tool patterned after the popular phpMyAdmin. phpPgAdmin differs from phpMyAdmin by including ways to manage PostgreSQL objects such as schemas, procedural languages, casts, operators, and so on. If you've used phpMyAdmin, you'll find phpPgAdmin to have the same look and feel.

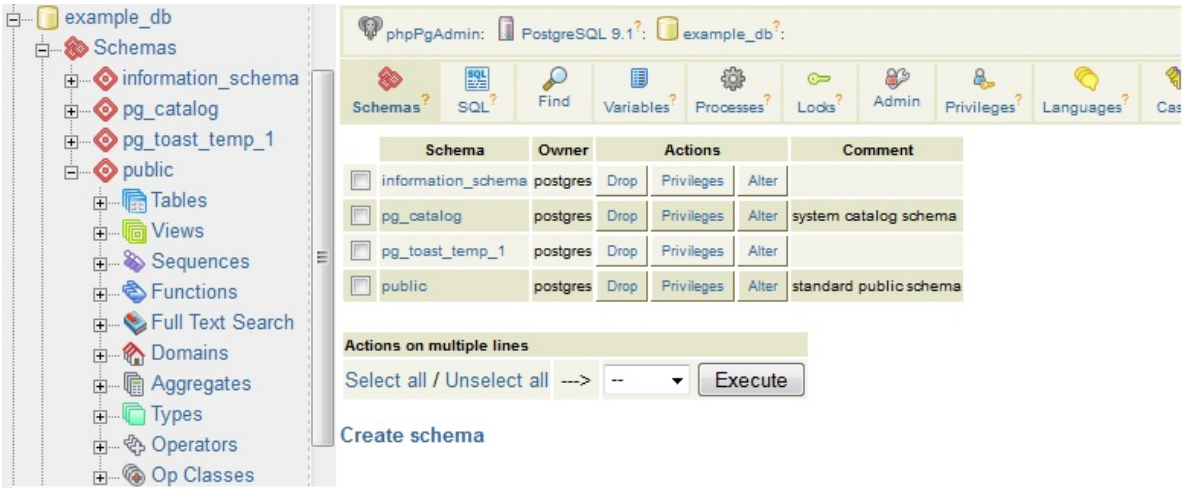

Figure 1-2. phpPgAdmin

Adminer

If you manage other databases besides PostgreSQL and are looking for a unified tool, Adminer might fit the bill. Adminer is a lightweight, open source PHP application with options for PostgreSQL, MySQL, SQLite, SQL Server, and Oracle, all delivered through a single interface.

One unique feature of Adminer we're impressed with is the relational diagrammer that can produce a schematic layout of your database schema, along with a linear representation of foreign key relationships. Another hassle-reducing feature is that you can deploy Adminer as a single PHP file.

Figure 1-3 is a screenshot of the login screen and a snippet from the diagrammer output. Many users stumble in the login screen of Adminer

because it doesn't include a separate text box for indicating the port number. If PostgreSQL is listening on the standard 5432 port, you need not worry. But if you use some other port, append the port number to the server name with a colon, as shown in Figure 1-3.

Adminer is sufficient for straightforward querying and editing, but because it's tailored to the lowest common denominator among database products, you won't find management applets that are specific to PostgreSQL for such tasks as creating new users, granting rights, or displaying permissions. Adminer also treats each schema as a separate database, which severely reduces the usefulness of the relational diagrammer if your relationships cross schema boundaries. If you're a DBA, stick to pgAdmin or psql.

Figure 1-3. Adminer

PostgreSQL Database Objects

So you installed PostgreSQL, fired up pgAdmin, and expanded its browse tree. Before you is a bewildering display of database objects, some familiar and some completely foreign. PostgreSQL has more database objects than most other relational database products (and that's before add-ons). You'll probably never touch many of these objects, but if you dream up something new, more likely than not it's already implemented using one of those esoteric objects. This book is not even going to attempt to describe all that you'll find in a standard PostgreSQL install. With PostgreSQL churning out

features at breakneck speed, we can't imagine any book that could possibly do this. We limit our quick overview to those objects that you should be familiar with:

Databases

Each PostgreSQL service houses many individual databases.

Schemas

Schemas are part of the ANSI SQL standard. They are the immediate next level of organization within each database. If you think of the database as a country, schemas would be the individual states (or provinces, prefectures, or departments, depending on the country). Most database objects first belong to a schema, which belongs to a database. When you create a new database, PostgreSQL automatically creates a schema named *public* to store objects that you create. If you have few tables, using public would be fine. But if you have thousands of tables, you should organize them into different schemas.

Tables

Tables are the workhorses of any database. In PostgreSQL, tables are first citizens of their respective schemas, which in turn are citizens of the database.

PostgreSQL tables have two remarkable talents: first, they are inheritable. Table inheritance streamlines your database design and can save you endless lines of looping code when querying tables with nearly identical structures. Second, whenever you create a table, PostgreSQL automatically creates an accompanying custom data type.

Views

Almost all relational database products offer views as a level of abstraction from tables. In a view, you can query multiple tables and present additional derived columns based on complex calculations. Views are generally read-only, but PostgreSQL allows you to update the underlying data by updating the view, provided that the view draws from

a single table. To update data from views that join multiple tables, you need to create a trigger against the view. Version 9.3 introduced materialized views, which cache data to speed up commonly used queries at the sacrifice of having the most up-to-date data. See "Materialized Views".

Extension

Extensions allow developers to package functions, data types, casts, custom index types, tables, attribute variables, etc., for installation or removal as a unit. Extensions are similar in concept to Oracle packages and have been the preferred method for distributing add-ons since PostgreSQL 9.1. You should follow the developer's instructions on how to install the extension files onto your server, which usually involves copying binaries into your PostgreSQL installation folders and then running a set of scripts. Once done, you must enable the extension for each database separately. You shouldn't enable an extension in your database unless you need it. For example, if you need advanced text search in only one database, enable `fuzzystrmatch` for that one only.

When you enable extensions, you choose the schemas where all constituent objects will reside. Accepting the default will place everything from the extension into the public schema, littering it with potentially thousands of new objects. We recommend that you create a separate schema that will house all extensions. For an extension with many objects, we suggest that you create a separate schema devoted entirely to it. Optionally, you can append the name of any schemas you add to the search_path variable of the database so you can refer to the function without having to prepend the schema name. Some extensions, especially ones that install a new procedural language (PL), will dictate the installation schema. For example, PL/V8 must be installed the pg_catalog schema.

Extensions may depend on other extensions. Prior to PostgreSQL 9.6, you had to know all dependent extensions and install them first. With 9.6, you simply need to add the `CASCADE` option and PostgreSQL will take care of

the rest. For example:

```
CREATE EXTENSION postgis_tiger_geocoder CASCADE;
```

first installs the dependent extensions postgis and fuzzystrmatch, if not present.

Functions

You can program your own custom functions to handle data manipulation, perform complex calculations, or wrap similar functionality. Create functions using PLs. PostgreSQL comes stocked with thousands of functions, which you can view in the postgres database that is part of every install. PostgreSQL functions can return scalar values, arrays, single records, or sets of records. Other database products refer to functions that manipulate data as stored procedures. PostgreSQL does not make this distinction.

Languages

Create functions using a PL. PostgreSQL installs three by default: SQL, PL/pgSQL, and C. You can easily install additional languages using the extension framework or the `CREATE PRODCEDURAL LANGUAGE` command. Languages currently in vogue are PL/Python, PL/V8 (JavaScript), and PL/R. We'll show you plenty of examples in Chapter 8.

Operators

Operators are nothing more than symbolically named aliases such as = or && for functions. In PostgreSQL, you can invent your own. This is often the case when you create custom data types. For example, if you create a custom data type of complex numbers, you'd probably want to also create addition operators (+,-,*,/) to handle arithmetic on them.

Foreign tables and foreign data wrappers

Foreign tables are virtual tables linked to data outside a PostgreSQL database. Once you've configured the link, you can query them like any

other tables. Foreign tables can link to CSV files, a PostgreSQL table on another server, a table in a different product such as SQL Server or Oracle, a NoSQL database such as Redis, or even a web service such as Twitter or Salesforce.

Foreign data wrappers (FDWs) facilitate the magic handshake between PostgreSQL and external data sources. FDW implementations in PostgreSQL follow the SQL/Management of External Data (MED) standard.

Many charitable programmers have already developed FDWs for popular data sources. You can try your hand at creating your own FDWs as well. (Be sure to publicize your success so the community can reap the fruits of your toil.) Install FDWs using the extension framework. Once installed, pgAdmin lists them under a node called Foreign Data Wrappers.

Triggers and trigger functions

You will find triggers in all enterprise-level databases; triggers detect data-change events. When PostgreSQL fires a trigger, you have the opportunity to execute trigger functions in response. A trigger can run in response to particular types of statements or in response to changes to particular rows, and can fire before or after a data-change event.

In pgAdmin, to see which table triggers, drill down to the table level. Pick the table of interest and look under *triggers*.

Create trigger functions to respond to firing of triggers. Trigger functions differ from regular functions in that they have access to special variables that store the data both before and after the triggering event. This allows you to reverse data changes made by the event during the execution of the trigger function. Because of this, trigger functions are often used to write complex validation routines that are beyond what can be implemented using check constraints.

Trigger technology is evolving rapidly in PostgreSQL. Starting in 9.0, a WITH clause lets you specify a boolean WHEN condition, which is tested to see whether the trigger should be fired. Version 9.0 also

introduced the UPDATE OF clause, which allows you to specify which column(s) to monitor for changes. When data in monitored columns changes, the trigger fires. In 9.1, a data change in a view can fire a trigger. Since 9.3, data definition language (DDL) events can fire triggers. For a list of triggerable DDL events, refer to the Event Trigger Firing Matrix. pgAdmin lists DDL triggers under the Event Triggers branch. Finally, as of version 9.4, you may place triggers against foreign tables.

Catalogs

Catalogs are system schemas that store PostgreSQL builtin functions and metadata. Every database contains two catalogs: *pg_catalog*, which holds all functions, tables, system views, casts, and types packaged with PostgreSQL; and *information_schema*, which offers views exposing metadata in a format dictated by the ANSI SQL standard.

PostgreSQL practices what it preaches. You will find that PostgreSQL itself is built atop a self-replicating structure. All settings to finetune servers are kept in system tables that you're free to query and modify. This gives PostgreSQL a level of extensibility (read hackability) impossible to attain by proprietary database products. Go ahead and take a close look inside the pg_catalog schema. You'll get a sense of how PostgreSQL is put together. If you have superuser privileges, you are at liberty to make updates to the pg_catalog directly (and screw things up royally).

The information_schema catalog is one you'll find in MySQL and SQL Server as well. The most commonly used views in the PostgreSQL information_schema are *columns*, which list all table columns in a database; *tables*, which list all tables (including views) in a database; and *views*, which list all views and the associated SQL to rebuild the view.

Types

Type is short for data type. Every database product and every programming language has a set of types that it understands: integers, characters, arrays, blobs, etc. PostgreSQL has composite types, which are

made up of other types. Think of complex numbers, polar coordinates, vectors, or tensors as examples.

Whenever you create a new table, PostgreSQL automatically creates a composite type based on the structure of the table. This allows you to treat table rows as objects in their own right. You'll appreciate this automatic type creation when you write functions that loop through tables. pgAdmin doesn't make the automatic type creation obvious because it does not list them under the types node, but rest assured that they are there.

Full text search

Full text search (FTS) is a natural language–based search. This kind of search has some "intelligence" built in. Unlike regular expression search, FTS can match based on the semantics of an expression, not just its syntactical makeup. For example, if you're searching for the word *running* in a long piece of text, you may end up with run, running, ran, runner, jog, sprint, dash, and so on. Three objects in PostgreSQL together support FTS: FTS configurations, FTS dictionaries, and FTS parsers. These objects exist to support the built-in Full Text Search engine packaged with PostgreSQL. For general use cases, the configurations, dictionaries, and parsers packaged with PostgreSQL are sufficient. But should you be working in a specific industry with specialized vocabulary and syntax rules such as pharmacology or organized crime, you can swap out the packaged FTS objects with your own. We cover FTS in detail in "Full Text Search".

Casts

Casts prescribe how to convert from one data type to another. They are backed by functions that actually perform the conversion. In PostgreSQL, you can create your own casts and override or enhance the default casting behavior. For example, imagine you're converting zip codes (which are five digits long in the US) to character from integer. You can define a custom cast that automatically prepends a zero when the zip is between 1000 and 9999.

Casting can be implicit or explicit. Implicit casts are automatic and usually expand from a more specific to a more generic type. When an implicit cast is not offered, you must cast explicitly.

Sequences

A sequence controls the autoincrementation of a serial data type. PostgresSQL automatically creates sequences when you define a serial column, but you can easily change the initial value, step, and next available value. Because sequences are objects in their own right, more than one table can share the same sequence object. This allows you to create a unique key value that can span tables. Both SQL Server and Oracle have sequence objects, but you must create them manually.

Rules

Rules are instructions to rewrite an SQL prior to execution. We're not going to cover rules as they've fallen out of favor because triggers can accomplish the same things.

For each object, PostgreSQL makes available many attribute variables that you can set. You can set variables at the server level, at the database level, at the function level, and so on. You may encounter the fancy term *GUC*, which stands for grand unified configuration, but it means nothing more than configuration settings in PostgreSQL.

What's New in Latest Versions of PostgreSQL?

Every September a new PostgreSQL is released. With each new release comes greater stability, heightened security, better performance—and avant-garde features. The upgrade process itself gets easier with each new version. The lesson here? Upgrade. Upgrade often. For a summary chart of key features added in each release, refer to the PostgreSQL Feature Matrix.

Why Upgrade?

If you're using PostgreSQL 9.1 or below, upgrade now! Version 9.1 retired to

end-of-life (EOL) status in September 2016. Details about PostgreSQL EOL policy can be found here: PostgreSQL Release Support Policy. EOL is not where you want to be. New security updates and fixes to serious bugs will no longer be available. You'll need to hire specialized PostgreSQL core consultants to patch problems or to implement workarounds—probably not a cheap proposition, assuming you can even locate someone willing to undertake the work.

Regardless of which major version you are running, you should always keep up with the latest micro versions. An upgrade from say, 9.1.17 to 9.1.21, requires no more than a file replacement and a restart. Micro versions only patch bugs. Nothing will stop working after a micro upgrade. Performing a micro upgrade can in fact save you much grief down the road.

Features Introduced in PostgreSQL 10

PostgreSQL 10 is the latest stable release and was released in October 2017. Starting with PostgreSQL 10, the PostgreSQL project adopted a new versioning convention. In prior versions, major versions got a minor version number bump. For example, PostgreSQL 9.6 introduced some major new features that were not in its PostgreSQL 9.5 predecessor. In contrast, starting with PostgreSQL 10, major releases will have the first digit bumped. So major changes to PostgreSQL 10 will be called PostgreSQL 11. This is more in line with what other database vendors follow, such as SQLite, SQL Server, and Oracle.

Here are the key new features in 10:

Query parallelization improvements

> There are new planner strategies for parallel queries: Parallel Bitmap Heap Scan, Parallel Index Scan, and others. These changes allow a wider range of queries to be parallelized for. See "Parallelized Queries".

Logical replication

> Prior versions of PostgreSQL had streaming replication that replicates the whole server cluster. Slaves in streaming replication were read-only and

could be used only for queries that don't change data. Nor could they have tables of their own. Logical replication provides two features that streaming replication did not have. You can now replicate just a table or a database (no need for the whole cluster); since you are replicating only part of the data, the slaves can have their own set of data that is not involved in replication.

Full text support for JSON and JSONB

In prior versions, to_tsvector would work only with plain text when generating a full text vector. Now to_tsvector can understand the json and jsonb types, ignoring the keys in JSON and including only the values in the vector. The ts_headline function for json and jsonb was also introduced. It highlights matches in a json document during a tsquery. Refer to "Full Text Support for JSON and JSONB".

ANSI standard XMLTABLE construct

XMLTABLE provides a simpler way of deconstructing XML into a standard table structure. This feature has existed for some time in Oracle and IBM DB2 databases. Refer to Example 5-41.

FDW push down aggregates to remote servers

The FDW API can now run aggregations such as COUNT(*) or SUM(*) on remote queries. postgres_fdw takes advantage of this new feature. Prior to postgres_fdw, any aggregation would require the local server to request all the data that needed aggregation and do the aggregation locally.

Declarative table partitioning

In prior versions, if you had a table you needed to partition but query as a single unit, you would utilize PostgreSQL table inheritance support. Using inheritance was cumbersome in that you had to write triggers to reroute data to a table PARTITION if adding to the parent table. PostgreSQL 10 introduces the PARTITION BY construct. PARTITION BY allows you to create a parent table with no data, but with a defined

PARTITION formula. Now you can insert data into the parent table without the need to define triggers. Refer to "Partitioned Tables".

Query execution

Various speedups have been added.

CREATE STATISTICS

New construct for creating statistics on multiple columns. Refer to Example 9-18.

IDENTITY

A new IDENTITY qualifier in DDL table creation and ALTER statements provides a more standards-compliant way to designate a table column as an auto increment. Refer to Example 6-2.

Features Introduced in PostgreSQL 9.6

PostgreSQL 9.6 was released in September 2016. PostgreSQL 9.6 is the last of the PostgreSQL 9+ series:

Query parallelization

Up to now, PostgreSQL could not take advantage of multiple processor cores. In 9.6, the PostgreSQL engine can distribute certain types of queries across multiple cores and processers. Qualified queries include those with sequential scans, some joins, and some aggregates. However, queries that involve changing data such as deletes, inserts, and updates are not parallelizable. Parallelization is a work in progress with the eventual hope that all queries will take advantage of multiple processor cores. See "Parallelized Queries".

Phrase full text search

Use the distance operator `<->` in a full text search query to indicate how far two words can be apart from each other and still be considered a match. In prior versions you could indicate only which words should be searched; now you can control the sequence of the words. See "Full Text

Search".

psql `\gexec` options

These read an SQL statement from a query and execute it. See "Dynamic SQL Execution".

postgres_fdw

Updates, inserts, and deletes are all much faster for simple cases. See *Depesz: Directly Modify Foreign Table* for details.

Pushed-down FDW joins

This is now supported by some FDWs. postgres_fdw supports this feature. When you join foreign tables, instead of retrieving the data from the foreign server and performing the join locally, FDW will perform the join remotely if foreign tables involved in the join are from the same foreign server and then retrieve the result set. This could lower the number of rows that have to come over from the foreign server, dramatically improving performance when joins eliminate many rows.

Features Introduced in PostgreSQL 9.5

Version 9.5 came out in January of 2016. Notable new features are as follows:

Improvements to foreign table architecture

A new `IMPORT FOREIGN SCHEMA` command allows for bulk creation of foreign tables from a foreign server. Foreign table inheritance means that a local table can inherit from foreign tables; foreign tables can inherit from local tables; and foreign tables can inherit from other foreign tables. You can also add constraints to foreign tables. See "Foreign Data Wrappers" and "Querying Other PostgreSQL Servers".

Using unlogged tables as a fast way to populate new tables

The downside is that unlogged tables would get truncated during a crash.

In prior versions, promoting an unlogged table to a logged table could not be done without creating a new table and repopulating the records. In 9.5, just use the `ALTER TABLE ... SET UNLOGGED` command.

Arrays in array_agg

The array_agg function accepts a set of values and combines them into a single array. Prior to 9.5, passing in arrays would throw an error. With 9.5, array_agg is smart enough to automatically construct multidimensional arrays for you. See Example 5-17.

Block range indexes (BRIN)

A new kind of index with smaller footprint than B-Tree and GIN. Under some circumstances BRIN can outperform the former two. See "Indexes".

Grouping sets, ROLLUP, AND CUBE SQL predicates

This feature is used in conjunction with aggregate queries to return additional subtotal rows. See "GROUPING SETS, CUBE, ROLLUP" for examples.

Index-only scans

These now support GiST indexes.

Insert and update conflict handling

Prior to 9.5, any inserts or updates that conflicted with primary key and check constraints would automatically fail. Now you have an opportunity to catch the exception and offer an alternative course, or to skip the records causing the conflict. See "UPSERTs: INSERT ON CONFLICT UPDATE".

Update lock failures

If you want to select and lock rows with the intent of updating the data, you can use `SELECT ... FOR UPDATE`. If you're unable to obtain the lock, prior to 9.5, you'd receive an error. With 9.5, you can add the `SKIP LOCKED` option to bypass rows for which you're unable to obtain locks.

Row-level security

You now have the ability to set visibility and updatability on rows of a table using policies. This is especially useful for multitenant databases or situations where security cannot be easily isolated by segmenting data into different tables.

Features Introduced in PostgreSQL 9.4

Version 9.4 came out in September 2014. Notable new features are as follows:

Materialized view enhancements

In 9.3, materialized views are inaccessible during a refresh, which could be a long time. This makes their deployment in a production undesirable. 9.4 eliminated the lock provided for materizalized views with a unique index.

New analytic functions to compute percentiles

percentile_disc (percentile discrete) and percentile_cont (percentile continuous) were added. They must be used with the special `WITHIN GROUP (ORDER BY ...)` construct. PostgreSQL vanguard Hubert Lubaczewski described their use in *Ordered Set Within Group Aggregates*. If you've ever looked for an aggregate median function in PostgreSQL, you didn't find it. Recall from your introduction to medians that the algorithm has an extra tie-breaker step at the end, making it difficult to program as an aggregate function. The new percentile functions approximate the true median with a "fast" median. We cover these two functions in more detail in "Percentiles and Mode".

Protection against updates in views

`WITH CHECK OPTION` clause added to the `CREATE VIEW` statement will block, update, or insert on the view if the resulting data would no longer be visible in the view. We demonstrate this feature in Example 7-3.

A new data type, `JSONB`

The JavaScript object notation binary type allows you to index a full JSON document and expedite retrieval of subelements. For details, see "JSON" and check out these blog posts: *Introduce jsonb: A Structured Format for Storing JSON* and *JSONB: Wildcard Query*.

Improved Generalized Inverted Index (GIN)

GIN was designed with FTS, trigrams, hstores, and JSONB in mind. Under many circumstances, you may choose GIN with its smaller footprint over B-Tree without loss in performance. Version 9.5 improved its query speed. Check out *GIN as a Substitute for Bitmap Indexes*.

More JSON functions

These are json_build_array, json_build_object, json_object, json_to_record, and json_to_recordset.

Expedited moves between tablespaces

You can now move all database objects from one tablespace to another by using the syntax `ALTER TABLESPACE old_space MOVE ALL TO new_space;`.

Row numbers in returned sets

You can add a row number for set-returning functions with the system column ordinality. This is particularly handy when converting denormalized data stored in arrays, hstores, and composite types to records. Here is an example using hstore:

```
SELECT ordinality, key, value
 FROM EACH('breed=>pug,cuteness=>high'::hstore) WITH ordinality;
```

Using SQL to alter system-configuration settings

The `ALTER system SET ...` construct allows you to set global system settings without editing the *postgresql.conf*, as detailed in "The

postgresql.conf File". This also means you can now programmatically change system settings, but keep in mind that PostgreSQL may require a restart for new settings to take effect.

Triggers

Version 9.4 lets you place triggers on foreign tables.

Better handling of unnesting

The unnest function predictably allocates arrays of different sizes into columns. Prior to 9.4, unnesting arrays of different sizes resulted in shuffling of columns in unexpected ways.

`ROWS FROM`

This construct allows the use of multiple set-returning functions in a series, even if they have an unbalanced number of elements in each set:

```
SELECT *
FROM ROWS FROM ( jsonb_each('{"a":"foo1","b":"bar"}'::jsonb),
                                 jsonb_each('{"c":"foo2"}'::jsonb)
)
                                 x (a1,a1_val,a2,a2_val);
```

Dynamic background workers

You can code these in C to do work that is not available through SQL or functions. A trivial example is available in the 9.4 source code in the *contrib/worker_spi* directory.

Database Drivers

Chances are that you're not using PostgreSQL in a vacuum. You need a database driver to interact with applications and other databases. PostgreSQL works with free drivers for many programming languages and tools. Moreover, various commercial organizations provide drivers with extra bells and whistles at modest prices. Here are some of the notable open source

drivers:

- PHP is a popular language for web development, and most PHP distributions include at least one PostgreSQL driver: the old pgsql driver or the newer pdo_pgsql. You may need to enable them in your *php.ini*.
- For Java developers, the JDBC driver keeps up with latest PostgreSQL versions. Download it from PostgreSQL.
- For .NET (both Microsoft or Mono), you can use the Npgsql driver. Both the source code and the binary are available for .NET Framework, Microsoft Entity Framework, and Mono.NET.
- If you need to connect from Microsoft Access, Excel, or any other products that support Open Database Connectivity (ODBC), download drivers from the PostgreSQL ODBC drivers site. You'll have your choice of 32-bit or 64-bit.
- LibreOffice 3.5 and later comes packaged with a native PostgreSQL driver. For OpenOffice and older versions of LibreOffice, you can use the JDBC driver or the SDBC driver. Learn more details from our article *OO Base and PostgreSQL*.
- Python has support for PostgreSQL via many database drivers. At the moment, psycopg2 is the most popular. Rich support for PostgreSQL is also available in the Django web framework. If you are looking for an object-relational mapper, SQL Alchemy is the most popular and is used internally by the Multicorn Foreign Data Wrapper.
- If you use Ruby, connect to PostgreSQL using rubygems pg.
- You'll find Perl's connectivity to PostgreSQL in the DBI and the DBD::Pg drivers. Alternatively, there's the pure Perl DBD::PgPP driver from CPAN.
- Node.js is a JavaScript framework for running scalable network programs. There are two PostgreSQL drivers currently: Node Postgres with optional native libpq bindings and pure JS (no compilation required) and Node-

DBI.

Where to Get Help

There will come a day when you need help. That day always arrives early; we want to point you to some resources now rather than later. Our favorite is the lively mailing list designed for helping new and old users with technical issues. First, visit PostgreSQL Help Mailing Lists. If you are new to PostgreSQL, the best list to start with is the PGSQL General Mailing List. If you run into what appears to be a bug in PostgreSQL, report it at PostgreSQL Bug Reporting.

Notable PostgreSQL Forks

The MIT/BSD-style licensing of PostgreSQL makes it a great candidate for forking. Various groups have done exactly that over the years. Some have contributed their changes back to the original project or funded PostgreSQL work. For list of forks, refer to PostgreSQL-derived databases.

Many popular forks are proprietary and closed source. Netezza, a popular database choice for data warehousing, was a PostgreSQL fork at inception. Similarly, the Amazon Redshift data warehouse is a fork of a fork of PostgreSQL. Amazon has two other offerings that are closer to standard PostgreSQL: Amazon RDS for PostgreSQL and Amazon Aurora for PostgreSQL. These stay in line with PostgreSQL versions in SQL syntax but with more management and speed features.

PostgreSQL Advanced Plus by EnterpriseDB is a fork that adds Oracle syntax and compatibility features to woo Oracle users. EnterpriseDB ploughs funding and development support back to the PostgreSQL community. For this, we’re grateful. Its Postgres Plus Advanced Server is fairly close to the most recent stable version of PostgreSQL.

Postgres-X2, Postgres-XL, and GreenPlum are three budding forks with open source licensing (although GreenPlum was closed source for a period). These

three target large-scale data analytics and replication.

Part of the reason for forking is to advance ahead of the PostgreSQL release cycle and try out new features that may or may not be of general interest. Many of the new features developed this way do find their way back into a later PostgreSQL core release. Such is the case with the multi-master bi-directional replication (BDR) fork developed by 2nd Quadrant. Pieces of BDR, such as the logical replication support, are beefing up the built-in replication functionality in PostgreSQL proper. Some of the parallelization work of Postgres-XL will also likely make it into future versions of PostgreSQL.

Citus is a project that started as a fork of PostgreSQL to support real-time big data and parallel queries. It has since been incorporated back and can be installed in PostgreSQL 9.5 as an extension.

Google Cloud SQL for PostgreSQL is a fairly recent addition by Google and is currently in beta.

Chapter 2. Database Administration

This chapter covers what we consider basic administration of a PostgreSQL server: managing roles and permissions, creating databases, installing extensions, and backing up and restoring data. Before continuing, you should have already installed PostgreSQL and have administration tools at your disposal.

Configuration Files

Three main configuration files control operations of a PostgreSQL server:

postgresql.conf

Controls general settings, such as memory allocation, default storage location for new databases, the IP addresses that PostgreSQL listens on, location of logs, and plenty more.

pg_hba.conf

Controls access to the server, dictating which users can log in to which databases, which IP addresses can connect, and which authentication scheme to accept.

pg_ident.conf

If present, this file maps an authenticated OS login to a PostgreSQL user. People sometimes map the OS root account to the PostgresSQL superuser account, *postgres*.

> NOTE
>
> PostgreSQL officially refers to users as *roles*. Not all roles need to have login

privileges. For example, group roles often do not. We use the term *user* to refer to a role with login privileges.

If you accepted default installation options, you will find these configuration files in the main PostgreSQL data folder. You can edit them using any text editor or the Admin Pack in pgAdmin. Instructions for editing with pgAdmin are in "Editing postgresql.conf and pg_hba.conf from pgAdmin3". If you are unable to find the physical location of these files, run the Example 2-1 query as a superuser while connected to any database.

Example 2-1. Location of configuration files

```
SELECT name, setting FROM pg_settings WHERE category = 'File Locations';
       name        |                 setting
-------------------+-----------------------------------------
 config_file       | /etc/postgresql/9.6/main/postgresql.conf
 data_directory    | /var/lib/postgresql/9.6/main
 external_pid_file | /var/run/postgresql/9.6-main.pid
 hba_file          | /etc/postgresql/9.6/main/pg_hba.conf
 ident_file        | /etc/postgresql/9.6/main/pg_ident.conf
(5 rows)
```

Making Configurations Take Effect

Some configuration changes require a PostgreSQL service restart, which closes any active connections from clients. Other changes require just a reload. New users connecting after a reload will receive the new setting. Extant users with active connections will not be affected during a reload. If you're not sure whether a configuration change requires a reload or restart, look under the context setting associated with a configuration. If the context is `postmaster`, you'll need a restart. If the context is `user`, a reload will suffice.

Reloading

A reload can be done in several ways. One way is to open a console window and run this command:

```
pg_ctl reload -D your_data_directory_here
```

If you have PostgreSQL installed as a service in RedHat Enterprise Linux, CentOS, or Ubuntu, enter instead:

```
service postgresql-9.5 reload
```

postgresql-9.5 is the name of your service. (For older versions of PostgreSQL, the service is sometimes called *postgresql* sans version number.)

You can also log in as a superuser to any database and execute the following SQL:

```
SELECT pg_reload_conf();
```

Finally, you can reload from pgAdmin; see "Editing postgresql.conf and pg_hba.conf from pgAdmin3".

Restarting

More fundamental configuration changes require a restart. You can perform a restart by stopping and restarting the postgres service (daemon). Yes, power cycling will do the trick as well.

You can't restart with a PostgreSQL command, but you can trigger a restart from the operating system shell. On Linux/Unix with a service, enter:

```
service postgresql-9.6 restart
```

For any PostgreSQL instance not installed as a service:

```
pg_ctl restart -D your_data_directory_here
```

On Windows you can also just click Restart on the PostgreSQL service in the Services Manager.

The postgresql.conf File

postgresql.conf controls the life-sustaining settings of the PostgreSQL server. You can override many settings at the database, role, session, and even function levels. You'll find many details on how to finetune your server by tweaking settings in the article Tuning Your PostgreSQL Server.

Version 9.4 introduced an important change: instead of editing *postgresql.conf* directly, you should override settings using an additional file called *postgresql.auto.conf*. We further recommend that you don't touch the *postgresql.conf* and place any custom settings in *postgresql.auto.conf*.

Checking postgresql.conf settings

An easy way to read the current settings without opening the configuration files is to query the view named *pg_settings*. We demonstrate in Example 2-2.

Example 2-2. Key settings

```
SELECT
    name,
    context ❶,
    unit ❷,
    setting, boot_val, reset_val ❸
FROM pg_settings
WHERE name IN ('listen_addresses','deadlock_timeout','shared_buffers',
    'effective_cache_size','work_mem','maintenance_work_mem')
ORDER BY context, name;
name                 | context    | unit | setting | boot_val  |
reset_val
---------------------+------------+------+-------- +-----------+---------
-
listen_addresses     | postmaster |      | *       | localhost | *
shared_buffers       | postmaster | 8kB  | 131584  | 1024      | 131584
deadlock_timeout     | superuser  | ms   | 1000    | 1000      | 1000
effective_cache_size | user       | 8kB  | 16384   | 16384     | 16384
maintenance_work_mem | user       | kB   | 16384   | 16384     | 16384
work_mem             | user       | kB   | 5120    | 1024      | 5120
```

❶ The context is the scope of the setting. Some settings have a wider effect than others, depending on their context.

User settings can be changed by each user to affect just that user's sessions. If set by the superuser, the setting becomes a default for all users who connect after a reload.

Superuser settings can be changed only by a superuser, and will apply to all users who connect after a reload. Users cannot individually override the setting.

Postmaster settings affect the entire server (postmaster represents the PostgreSQL service) and take effect only after a restart.

Settings with user or superuser context can be set for a specific database, user, session, and function level. For example, you might want to set work_mem higher for an SQL guru-level user who writes mind-boggling queries. Similarly, if you have one function that is sort-intensive, you could raise work_mem just for it. Settings set at database, user, session, and function levels do not require a reload. Settings set at the database level take effect on the next connect to the database. Settings set for the session or function take effect right away.

❷ Be careful checking the units of measurement used for memory. As you can see in Example 2-2, some are reported in 8-KB blocks and some just in kilobytes. Regardless of how a setting displays, you can use any unit of choice when setting; 128 MB is a versatile choice for most memory settings.

Showing units as 8 KB is annoying at best and is destabilizing at worst. The `SHOW` command in SQL offers display settings in labeled and more intuitive units. For example, running:

```
SHOW shared_buffers;
```

returns `1028MB`. Similarly, running:

```
SHOW deadlock_timeout;
```

returns `1s`. If you want to see the units for all settings, enter `SHOW ALL`.

❸ `setting` is the current setting; `boot_val` is the default setting; `reset_val` is the new setting if you were to restart or reload the server. Make sure that `setting` and `reset_val match` after you make a change. If not, the server needs a restart or reload.

New in version 9.5 is a system view called *pg_file_settings*, which you can use to query settings. Its output lists the source file where the settings can be found. The *applied* tells you whether the setting is in effect; if the setting has an `f` in that column you need to reload or restart to make it take effect. In cases where a particular setting is present in both *postgresql.conf* and *postgresql.auto.conf*, the *postgresql.auto.conf* one will take precedent and you'll see the other files with applied set to false (f). The applied is shown in Example 2-3.

Example 2-3. Querying pg_file_settings

```
SELECT name, sourcefile, sourceline, setting, applied
FROM pg_file_settings
WHERE name IN ('listen_addresses','deadlock_timeout','shared_buffers',
    'effective_cache_size','work_mem','maintenance_work_mem')
ORDER BY name;
name                 | sourcefile                    | sourceline |
setting | applied
---------------------+-------------------------------+------------+------
---+--------
effective_cache_size | E:/data96/postgresql.auto.conf|         11 | 8GB
| t
listen_addresses     | E:/data96/postgresql.conf     |         59 | *
| t
maintenance_work_mem | E:/data96/postgresql.auto.conf|          3 | 16MB
| t
shared_buffers       | E:/data96/postgresql.conf     |        115 | 128MB
| f
shared_buffers       | E:/data96/postgresql.auto.conf|          5 |
131584  | t
```

Pay special attention to the following network settings in *postgresql.conf* or *postgresql.auto.conf*, because an incorrect entry here will prevent clients from connecting. Changing their values requires a service restart:

listen_addresses

Informs PostgreSQL which IP addresses to listen on. This usually defaults to local (meaning a socket on the local system), or localhost, meaning the IPv6 or IPv4 localhost IP address. But many people change the setting to *, meaning all available IP addresses.

port

Defaults to 5432. You may wish to change this *well-known port* to something else for security or if you are running multiple PostgreSQL services on the same server.

max_connections

The maximum number of concurrent connections allowed.

log_destination

This setting is somewhat a misnomer. It specifies the format of the logfiles rather than their physical location. The default is *stderr*. If you intend to perform extensive analysis on your logs, we suggest changing it to *csvlog*, which is easier to export to third-party analytic tools. Make sure you have the logging_collection set to on if you want logging.

The following settings affect performance. Defaults are rarely the optimal value for your installation. As soon as you gain enough confidence to tweak configuration settings, you should tune these values:

shared_buffers

Allocated amount of memory shared among all connections to store recently accessed pages. This setting profoundly affects the speed of your queries. You want this setting to be fairly high, probably as much as 25% of your RAM. However, you'll generally see diminishing returns after more than 8 GB. Changes require a restart.

effective_cache_size

An estimate of how much memory PostgreSQL expects the operating system to devote to it. This setting has no effect on actual allocation, but the query planner figures in this setting to guess whether intermediate

steps and query output would fit in RAM. If you set this much lower than available RAM, the planner may forgo using indexes. With a dedicated server, setting the value to half of your RAM is a good starting point. Changes require a reload.

work_mem

Controls the maximum amount of memory allocated for each operation such as sorting, hash join, and table scans. The optimal setting depends on how you're using the database, how much memory you have to spare, and whether your server is dedicated to PostgreSQL. If you have many users running simple queries, you want this setting to be relatively low to be democratic; otherwise, the first user may hog all the memory. How high you set this also depends on how much RAM you have to begin with. A good article to read for guidance is Understanding work_mem. Changes require a reload.

maintenance_work_mem

The total memory allocated for housekeeping activities such as vacuuming (pruning records marked for deletion). You shouldn't set it higher than about 1 GB. Reload after changes.

max_parallel_workers_per_gather

This is a new setting introduced in 9.6 for parallelism. The setting determines the maximum parallel worker threads that can be spawned for each gather operation. The default setting is 0, which means parallelism is completely turned off. If you have more than one CPU core, you will want to elevate this. Parallel processing is new in version 9.6, so you may have to experiment with this setting to find what works best for your server. Also note that the number you have here should be less than `max_worker_processes`, which defaults to 8 because the parallel background worker processes are a subset of the maximum allowed processes.

In version 10, there is an additional setting called `max_parallel_workers`, which controls the subset of

max_worker_processes allocated for parallelization.

Changing the postgresql.conf settings

PostgreSQL 9.4 introduced the ability to change settings using the *ALTER SYSTEM* SQL command. For example, to set the work_mem globally, enter the following:

```
ALTER SYSTEM SET work_mem = '500MB';
```

This command is wise enough to not directly edit *postgres.conf* but will make the change in *postgres.auto.conf*.

Depending on the particular setting changed, you may need to restart the service. If you just need to reload it, here's a convenient command:

```
SELECT pg_reload_conf();
```

If you have to track many settings, consider organizing them into multiple configuration files and then linking them back using the *include* or *include_if_exists* directive within the *postgresql.conf*. The exact syntax is as follows:

```
include 'filename'
```

The filename argument can be an absolute path or a relative path from the *postgresql.conf* file.

"I edited my postgresql.conf and now my server won't start."

The easiest way to figure out what you screwed up is to look at the logfile, located at the root of the data folder, or in the *pg_log* subfolder. Open the latest file and read what the last line says. The error raised is usually self-explanatory.

A common culprit is setting shared_buffers too high. Another suspect is an old *postmaster.pid* left over from a failed shutdown. You can safely delete

this file, located in the data cluster folder, and try restarting again.

The pg_hba.conf File

The *pg_hba.conf* file controls which IP addresses and users can connect to the database. Furthermore, it dictates the authentication protocol that the client must follow. Changes to the file require at least a reload to take effect. A typical *pg_hba.conf* looks like Example 2-4.

Example 2-4. Sample pg_hba.conf

```
# TYPE  DATABASE USER ADDRESS         METHOD
host    all      all  127.0.0.1/32    ident ❶
host    all      all  ::1/128         trust ❷
host    all      all  192.168.54.0/24 md5 ❸
hostssl all      all  0.0.0.0/0       md5 ❹

# TYPE DATABASE    USER     ADDRESS      METHOD
# Allow replication connections from localhost,
# by a user with replication privilege. ❺
#host  replication postgres 127.0.0.1/32 trust
#host  replication postgres ::1/128      trust
```

❶ Authentication method. The usual choices are ident, trust, md5, peer, and password.

❷ IPv6 syntax for defining network range. This applies only to servers with IPv6 support and may prevent *pg_hba.conf* from loading if you add this section without actually having IPv6 networking enabled on the server.

❸ IPv4 syntax for defining network range. The first part is the network address followed by the bit mask; for instance: 192.168.54.0/24. PostgreSQL will accept connection requests from any IP address within the range.

❹ SSL connection rule. In our example, we allow anyone to connect to our server outside of the allowed IP range as long as they can connect using SSL.

SSL configuration settings can be found in *postgres.conf* or *postgres.auto.conf*: `ssl`, `ssl_cert_file`, `ssl_key_file`. Once the server confirms that the client is able to support SSL, it will honor the

connection request and all transmissions will be encrypted using the key information.

❺ Range of IP addresses allowed to replicate with this server.

For each connection request, *pg_hba.conf* is checked from the top down. As soon as a rule granting access is encountered, a connection is allowed and the server reads no further in the file. As soon as a rule rejecting access is encountered, the connection is denied and the server reads no further in the file. If the end of the file is reached without any matching rules, the connection is denied. A common mistake people make is to put the rules in the wrong order. For example, if you added `0.0.0.0/0 reject` before `127.0.0.1/32 trust`, local users won't be able to connect, even though a rule is in place allowing them to.

New in version 10 is the `pg_hba_file_rules` system view that lists all the contents of the *pg_hba.conf* file.

"I edited my pg_hba.conf and now my server is broken."

Don't worry. This happens quite often, but is easy to recover from. This error is generally caused by typos or by adding an unavailable authentication scheme. When the postgres service can't parse *pg_hba.conf*, it blocks all access just to be safe. Sometimes, it won't even start up. The easiest way to figure out what you did wrong is to read the logfile located in the root of the data folder or in the *pg_log* subfolder. Open the latest file and read the last line. The error message is usually self-explanatory. If you're prone to slippery fingers, back up the file prior to editing.

Authentication methods

PostgreSQL gives you many choices for authenticating users—probably more than any other database product. Most people are content with the popular ones: trust, peer, ident, md5, and password. And don't forget about reject, which immediately denies access. Also keep in mind that *pg_hba.conf* offers settings at many other levels as the gatekeeper to the entire PostgreSQL server. Users or devices must still satisfy role and database

access restrictions after being admitted by *pg_hba.conf*.

We describe the common authentication methods here:

trust

This is the least secure authentication, essentially no password is needed. As long as the user and database exist in the system and the request comes from an IP within the allowed range, the user can connect. You should implement trust only for local connections or private network connections. Even then it's possible for someone to spoof IP addresses, so the more security-minded among us discourage its use entirely. Nevertheless, it's the most common for PostgreSQL installed on a desktop for single-user local access where security is not a concern.

md5

Very common, requires an md5-encrypted password to connect.

password

Uses clear-text password authentication.

ident

Uses *pg_ident.conf* to check whether the OS account of the user trying to connect has a mapping to a PostgreSQL account. The password is not checked. ident is not available on Windows.

peer

Uses the OS name of the user from the kernel. It is available only for Linux, BSD, macOS, and Solaris, and only for local connections on these systems.

cert

Stipulates that connections use SSL. The client must have a registered certificate. cert uses an ident file such as *pg_ident* to map the certificate to a PostgreSQL user and is available on all platforms where SSL connection is enabled.

More esoteric options abound, such as gss, radius, ldap, and pam. Some may not always be installed by default.

You can elect more than one authentication method, even for the same database. Keep in mind that *pg_hba.conf* is processed from top to bottom.

Managing Connections

More often than not, someone else (never you, of course) will execute an inefficient query that ends up hogging resources. They could also run a query that's taking much longer than what they have patience for. Cancelling the query, terminating the connection, or both will put an end to the offending query.

Cancelling and terminating are far from graceful and should be used sparingly. Your client application should prevent queries from going haywire in the first place. Out of politeness, you probably should alert the connected role that you're about to terminate its connection, or wait until after hours to do the dirty deed.

There are few scenarios where you should cancel all active update queries: before backing up the database and before restoring the database.

To cancel running queries and terminate connections, follow these steps:

1. Retrieve a listing of recent connections and process IDs (PIDs):

   ```
   SELECT * FROM pg_stat_activity;
   ```

   pg_stat_activity is a view that lists the last query running on each connection, the connected user (username), the database (datname) in use, and the start times of the queries. Review the list to identify the PIDs of connections you wish to terminate.

2. Cancel active queries on a connection with PID *1234*:

   ```
   SELECT pg_cancel_backend(1234);
   ```

This does not terminate the connection itself, though.

3. Terminate the connection:

```
SELECT pg_terminate_backend(1234);
```

You may need to take the additional step of terminating the client connection. This is especially important prior to a database restore. If you don't terminate the connection, the client may immediately reconnect after restore and run the offending query anew. If you did not already cancel the queries on the connection, terminating the connection will cancel all of its queries.

PostgreSQL lets you embed functions within a regular SELECT statement. Even though pg_terminate_backend and pg_cancel_backend act on only one connection at a time, you can kill multiple connections by wrapping them in a SELECT. For example, let's suppose you want to kill all connections belonging to a role with a single blow. Run this SQL command:

```
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE usename = 'some_role';
```

You can set certain operational parameters at the server, database, user, session, or function level. Any queries that exceed the parameter will automatically be cancelled by the server. Setting a parameter to 0 disables the parameter:

deadlock_timeout

This is the amount of time a deadlocked query should wait before giving up. This defaults to 1000 ms. If your application performs a lot of updates, you may want to increase this value to minimize contention.

Instead of relying on this setting, you can include a NOWAIT clause in your update SQL: `SELECT FOR UPDATE NOWAIT ...` .

The query will be automatically cancelled upon encountering a deadlock.

In PostgreSQL 9.5, you have another choice: `SELECT FOR UPDATE SKIP LOCKED` will skip over locked rows.

statement_timeout

This is the amount of time a query can run before it is forced to cancel. This defaults to 0, meaning no time limit. If you have long-running functions that you want cancelled if they exceed a certain time, set this value in the definition of the function rather than globally. Cancelling a function cancels the query and the transaction that's calling it.

lock_timeout

This is the amount of time a query should wait for a lock before giving up, and is most applicable to update queries. Before data updates, the query must obtain an exclusive lock on affected records. The default is 0, meaning that the query will wait infinitely. This setting is generally used at the function or session level. lock_timeout should be lower than statement_timeout, otherwise statement_timeout will always occur first, making lock_timeout irrelevant.

idle_in_transaction_session_timeout

This is the amount of time a transaction can stay in an idle state before it is terminated. This defaults to 0, meaning it can stay alive infinitely. This setting is new in PostgreSQL 9.6. It's useful for preventing queries from holding on to locks on data indefinitely or eating up a connection.

Check for Queries Being Blocked

The `pg_stat_activity` view has changed considerably since version 9.1 with the renaming, dropping, and addition of new columns. Starting from version 9.2, `procpid` was renamed to `pid`.

`pg_stat_activity` changed in PostgreSQL 9.6 to provide more detail about waiting queries. In prior versions of PostgreSQL, there was a field called `waiting` that could take the value `true` or `false`. `true` denoted a query that

was being blocked waiting some resource, but the resource being waited for was never stated. In PostgreSQL 9.6, `waiting` was removed and replaced with `wait_event_type` and `wait_event` to provide more information about what resource a query was waiting for. Therefore, prior to PostgreSQL 9.6, use `waiting = true` to determine what queries are being blocked. In PostgreSQL 9.6 or higher, use `wait_event IS NOT NULL`.

In addition to the change in structure, PostgreSQL 9.6 will now track additional wait locks that did not get set to `waiting=true` in prior versions. As a result, you may find lighter lock waits being listed for queries than you saw in prior versions. For a list of different wait_event types, refer to PostgreSQL Manual: wait_event names and types.

Roles

PostgreSQL handles credentialing using *roles*. Roles that can log in are called *login roles*. Roles can also be members of other roles; the roles that contain other roles are called *group roles*. (And yes, group roles can be members of other group roles and so on, but don't go there unless you have a knack for hierarchical thinking.) Group roles that can log in are called *group login roles*. However, for security, group roles generally cannot log in. A role can be designated as a *superuser*. These roles have unfettered access to the PostgreSQL service and should be assigned with discretion.

> WARNING
>
> Recent versions of PostgreSQL no longer use the terms *users* and *groups*. You will still run into these terms; just know that they mean login roles and group roles, respectively. For backward compatibility, CREATE USER and CREATE GROUP still work in current versions, but shun them and use CREATE ROLE instead.

Creating Login Roles

When you initialize the data cluster during setup, PostgreSQL creates a single login role with the name *postgres*. (PostgreSQL also creates a namesake database called postgres.) You can bypass the password setting by mapping an OS root user to the new role and using `ident`, `peer`, or `trust` for authentication. After you've installed PostgreSQL, before you do anything else, you should log in as postgres and create other roles. pgAdmin has a graphical section for creating user roles, but if you want to create one using SQL, execute an SQL command like the one shown in Example 2-5.

Example 2-5. Creating login roles

```
CREATE ROLE leo LOGIN PASSWORD 'king' VALID UNTIL 'infinity' CREATEDB;
```

Specifying VALID UNTIL is optional. If omitted, the role remains active indefinitely. CREATEDB grants database creation privilege to the new role.

To create a user with superuser privileges, follow Example 2-6. Naturally, you must be a superuser to create other superusers.

Example 2-6. Creating superuser roles

```
CREATE ROLE regina LOGIN PASSWORD 'queen' VALID UNTIL '2020-1-1 00:00'
SUPERUSER;
```

Both of the previous examples create roles that can log in. To create roles that cannot log in, omit the LOGIN PASSWORD clause.

Creating Group Roles

Group roles generally cannot log in. Rather, they serve as containers for other roles. This is merely a best-practice suggestion. Nothing stops you from creating a role that can log in as well as contain other roles.

Create a group role using the following SQL:

```
CREATE ROLE royalty INHERIT;
```

Note the use of the modifier *INHERIT*. This means that any member of royalty will automatically inherit privileges of the royalty role, except for the superuser privilege. For security, PostgreSQL never passes down the

superuser privilege. INHERIT is the default, but we recommend that you always include the modifier for clarity.

To refrain from passing privileges from the group to its members, create the role with the *NOINHERIT* modifier.

To add members to a group role, you would do:

```
GRANT royalty TO leo;
GRANT royalty TO regina;
```

Some privileges can't be inherited. For example, although you can create a group role that you mark as superuser, this doesn't make its member roles superusers. However, those users can *impersonate* their group role by using the SET ROLE command, thereby gaining superuser privileges for the duration of the session. For example:

Let's give the royalty role superuser rights with the command:

```
ALTER ROLE royalty SUPERUSER;
```

Although `leo` is a member of the royalty group and he inherits most rights of royalty, when he logs in, he still will not have superuser rights. He can gain superuser rights by doing:

```
SET ROLE royalty;
```

His superuser rights will last only for his current session.

This feature, though peculiar, is useful if you want to prevent yourself from unintentionally doing superuser things while you are logged in.

`SET ROLE` is a command available to all users, but a more powerful command called `SET SESSION AUTHORIZATION` is available to people who log in as superusers. In order to understand the differences, we'll first introduce two global variables that PostgreSQL has called: `current_user` and `session_user`. You can see these values when you log in by running the

SQL statement:

```
SELECT session_user, current_user;
```

When you first log in, the values of these two variables are the same. `SET ROLE` changes the `current_user`, while `SET SESSION AUTHORIZATION` changes both the `current_user` and `session_user` variables.

Here are the salient properties of `SET ROLE`:

- SET ROLE does not require superuser rights.
- SET ROLE changes the current_user variable, but not the session_user variable.
- A session_user that has superuser rights can SET ROLE to any other role.
- Nonsuperusers can SET ROLE only to the role the session_user is or the roles the session_user belongs to.
- When you do SET ROLE you gain all privileges of the impersonated user except for SET SESSION_AUTHORIZATION and SET ROLE.

A more powerful command, `SET SESSION AUTHORIZATION`, is available as well. Key features of SET SESSION AUTHORIZATION are as follows:

- Only a user that logs in as a superuser has permission to do SET SESSION AUTHORIZATION to another role.
- The SET SESSION AUTHORIZATION privilege is in effect for the life of the session, meaning that even if you SET SESSION AUTHORIZATION to a user that is not a superuser, you still have the SET SESSION AUTHORIZATION privilege for the life of your session.
- SET SESSION AUTHORIZATION changes the values of the current_user and session_user variables to those of the user being impersonated.
- A session_user that has superuser rights can SET ROLE to any other role.

We'll do a set of exercises that illustrate the differences between `SET ROLE` and `SET SESSION AUTHORIZATION` by first logging in as *leo* and then running the code in Example 2-7.

Example 2-7. SET ROLE and SET AUTHORIZATION

```
SELECT session_user, current_user;
 session_user | current_user
--------------+--------------
 leo          | leo
(1 row)
SET SESSION AUTHORIZATION regina;
ERROR:  permission denied to set session authorization
SET ROLE regina;
ERROR:  permission denied to set role "regina"
ALTER ROLE leo SUPERUSER;
ERROR:  must be superuser to alter superusers
SET ROLE royalty;
SELECT session_user, current_user;
 session_user | current_user
--------------+--------------
 leo          | royalty
(1 row)
SET ROLE regina;
ERROR:  permission denied to set role "regina"
ALTER ROLE leo SUPERUSER;
SET ROLE regina;
SELECT session_user, current_user;
 session_user | current_user
--------------+--------------
 leo          | regina
(1 row)
SET SESSION AUTHORIZATION regina;
ERROR:  permission denied to set session authorization
-- After ending session and logging back in as leo
SELECT session_user, current_user;
SET SESSION AUTHORIZATION regina;
SELECT session_user, current_user;
session_user | current_user
--------------+--------------
leo | leo
(1 row)
SET SESSION AUTHORIZATION
```

```
session_user | current_user
--------------+--------------
regina | regina
(1 row)
```

In Example 2-7 leo was unable to use SET SESSION AUTHORIZATION because he's not a superuser. He was also unable to SET ROLE to regina because he is not in the regina group. However, he was able to `SET ROLE royalty` since he is a member of the royalty group (he's a king consort). Even though royalty has superuser rights, he still wasn't able to impersonate the queen, regina, because his SET ROLE abilities are still based on being the powerless leo. Since royalty is a group that has superuser rights, he was able to promote his own account leo to be a superuser. Once leo is promoted to power, he can then impersonate regina. He is now able to completely take over her session_user and current_user persona with SET SESSION AUTHORIZATION.

Database Creation

The minimum SQL command to create a database is:

```
CREATE DATABASE mydb;
```

This creates a copy of the template1 database. Any role with CREATEDB privilege can create new databases.

Template Databases

A template database is, as the name suggests, a database that serves as a skeleton for new databases. When you create a new database, PostgreSQL copies all the database settings and data from the template database to the new database.

The default PostgreSQL installation comes with two template databases: *template0* and *template1*. If you don't specify a template database to follow when you create a database, *template1* is used.

> **WARNING**
>
> You should never alter template0 because it is the immaculate model that you'll need to copy from if you screw up your templates. Make your customizations to template1 or a new template database you create. You can't change the encoding and collation of a database you create from template1 or any other template database you create. So if you need a different encoding or collation from those in template1, create the database from template0.

The basic syntax to create a database modeled after a specific template is:

```
CREATE DATABASE my_db TEMPLATE my_template_db;
```

You can pick any database to serve as the template. This could come in quite handy when making replicas. You can also mark any database as a template database. Once you do, the database is no longer editable and deletable. Any role with the CREATEDB privilege can use a template database. To make any database a template, run the following SQL as a superuser:

```
UPDATE pg_database SET datistemplate = TRUE WHERE datname = 'mydb';
```

If ever you need to edit or drop a template database, first set the *datistemplate* attribute to FALSE. Don't forget to change the value back after you're done with edits.

Using Schemas

Schemas organize your database into logical groups. If you have more than two dozen tables in your database, consider cubbyholing them into schemas. Objects must have unique names within a schema but need not be unique across the database. If you cram all your tables into the default public schema, you'll run into name clashes sooner or later. It's up to you how to organize your schemas. For example, if you are an airline, you can place all tables of planes you own and their maintenance records into a planes schema. Place all your crew and staff into an employees schema and place all

passenger-related information into a passengers schema.

Another common way to organize schemas is by roles. We found this to be particularly handy with applications that serve multiple clients whose data must be kept separate.

Suppose that you started a dog beauty management business (doggie spa). You start with a table in public called dogs to track all the dogs you hope to groom. You convince your two best friends to become customers. Whimsical government privacy regulation passes, and now you have to put in iron-clad assurances that one customer cannot see dog information from another. To comply, you set up one schema per customer and create the same dogs table in each as follows:

```
CREATE SCHEMA customer1;
CREATE SCHEMA customer2;
```

You then move the dog records into the schema that corresponds with the client. The final touch is to create different login roles for each schema with the same name as the schema. Dogs are now completely isolated in their respective schemas. When customers log in to your database to make appointments, they will be able to access only information pertaining to their own dogs.

Wait, it gets better. Because we named our roles to match their respective schemas, we're blessed with another useful technique. But we must first introduce the *search_path* database variable.

As we mentioned earlier, object names must be unique within a schema, but you can have same-named objects in different schemas. For example, you have the same table called dogs in all 12 of your schemas. When you execute something like `SELECT * FROM dogs`, how does PostgreSQL know which schema you're referring to? The simple answer is to always prepend the schema name onto the table name with a dot, such as in `SELECT * FROM customer1.dogs`. Another method is to set the search_path variable to be something like `customer1, public`. When the query executes, the planner searches for the dogs table first in the customer1 schema. If not found, it

continues to the public schema and stops there.

PostgreSQL has a little-known variable called *user* that retrieves the role currently logged in. `SELECT user` returns this name. `user` is just an alias for `current_user`, so you can use either.

Recall how we named our customers' schemas to be the same as their login roles. We did this so that we can take advantage of the default search path set in *postgresql.conf*:

```
search_path = "$user", public;
```

Now, if role customer1 logs in, all queries will first look in the customer1 schema for the tables before moving to public. Most importantly, the SQL remains the same for all customers. Even if the business grows to have thousands or hundreds of thousands of dog owners, none of the SQL scripts need to change. Commonly shared tables such as common lookup tables can be put in the public schema.

Another practice that we strongly encourage is to create schemas to house extensions ("Step 2: Installing into a database"). When you install an extension, new tables, functions, data types, and plenty of other relics join your server. If they all swarm into the public schema, it gets cluttered. For example, the entire PostGIS suite of extensions will together add thousands of functions. If you've already created a few tables and functions of your own in the public schema, imagine how maddening it would be to scan a list of tables and functions trying to find your own among the thousands.

Before you install any extensions, create a new schema:

```
CREATE SCHEMA my_extensions;
```

Then add your new schema to the search path:

```
ALTER DATABASE mydb SET search_path='$user', public, my_extensions;
```

When you install extensions, be sure to indicate your new schema as their